

# Laboratorio di programmazione per sistemi ciberfisici

## 15. Allocazione dinamica di memoria

*Enrico Martini*

*April 20, 2026*

## Memoria statica vs dinamica

### Allocazione statica (**Stack**):

- ▶ Contiene variabili locali e parametri delle funzioni
- ▶ Allocazione e deallocazione automatiche
- ▶ Molto veloce
- ▶ Dimensione limitata

### Allocazione dinamica (**Heap**):

- ▶ Allocazione e deallocazione manuale
- ▶ Dimensione flessibile a seconda delle esigenze
- ▶ Accesso a memoria più lento
- ▶ Funzioni in `stdlib.h`

## Allocazione di memoria sullo heap

La funzione `malloc` richiede il **numero di byte necessari** in input e ritorna il **puntatore a una zona di memoria** (chiamata heap) della dimensione selezionata:

```
1 void* malloc(int size){ ... }    // Chiede un numero e ritorna un puntatore
```

Per non doversi ricordare la dimensione di ogni tipo di dato, si utilizza l'operatore `sizeof`:

```
1 sizeof(int);           // Dimensione del tipo "int"  
2 sizeof(struct date);   // Dimensione di una struttura  
3 int x[10]; sizeof(x);  // Dimensione di un intero array
```

## Allocazione di memoria sullo heap

Esempio di utilizzo:

```
1 int *p = (int *) malloc(1000 * sizeof(int));
```

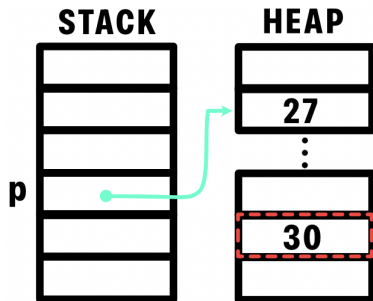
Caratteristiche:

- ▶ La funzione **restituisce l'indirizzo di memoria** da cui parte la zona di memoria richiesta, oppure NULL se allocazione impossibile (es. memoria insufficiente)
- ▶ Occorre **cast esplicito** a tipo di puntatore, perchè inizialmente è void\*

# Memory Leak

Perdita di accesso a spazio  
allocato in memoria:

```
1 int *p;  
2 p = (int *) malloc(sizeof(int));  
3 *p = 30;  
4 p = (int *) malloc(sizeof(int));  
5 *p = 27;
```



## Deallocazione di memoria sullo heap

- ▶ A differenza di moderni linguaggi di programmazione (es: Python), C non dispone di un garbage collector
- ▶ Quando si ha finito di utilizzare una zona di memoria nello heap, è **necessario restituirla** al sistema con la funzione `free()`:

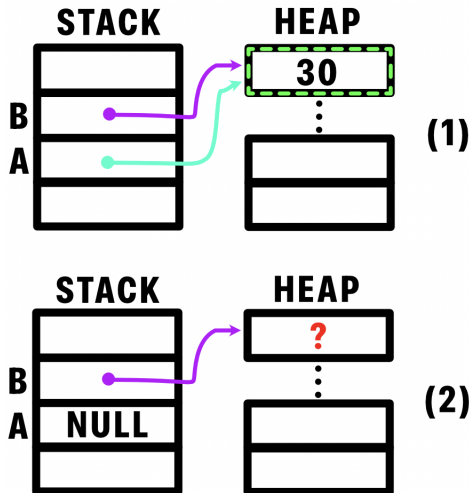
```
1 free(puntatore);
```

- ▶ Liberare la memoria è fondamentale per **evitare memory leaks**
- ▶ Bisogna assicurarsi che **ogni chiamata a `malloc` abbia la chiamata a `free`**
- ▶ Liberare più volte un'area di memoria (**doppia liberazione**) può portare a comportamenti indefiniti, come crash del programma.
- ▶ Si consiglia di assegnare `NULL` al puntatore liberato

## Dangling Pointer

Accesso a zona di memoria deallocata:

```
1  int *A, *B;  
2  A = (int *) malloc(sizeof(int));  
3  B = A;  
4  *A = 30;    // Figura 1  
5  free(A);  
6  A = NULL;   // Figura 2  
7  *B = 10;
```



## Funzioni utili

Esistono strumenti che aiutano a rilevare problemi di gestione della memoria come `valgrind`:

```
1 valgrind --leak-check=full ./a.out
```

Esistono due altre funzioni:

- ▶ `calloc(n,size)`: richiede il numero di elementi da riservare e la dimensione di ciascun elemento in byte. Restituisce un puntatore.
- ▶ `realloc()`: utilizzata per comprimere o espandere aree di memoria precedentemente allocate.



## Esercizio 1

Scrivere un programma che allochi un intero nello heap, assegni 1 e ne stampi il valore.

## Esercizio 2

Scrivere un programma che allochi un array di 10 elementi nello heap, assegni 1 a tutti e ne stampi il valore.

## Esercizio 3

Creare una struttura dati semplice (es. struct Point) e allocarne una istanza dinamicamente.

## Esercizio 4

Scrivere una funzione che concatena due stringhe dinamiche e restituire una nuova stringa.

## Esercizio 5

**Definizione PILA:** struttura dati in cui si accede con modalità LIFO (Last In First Out).

Implementare una pila utilizzando strutture e puntatori. Ciascun elemento della pila è una struttura ricorsiva formata da un valore (intero) e da un puntatore alla struttura rappresentante l'elemento successivo. La fine dello stack va identificata con un puntatore NULL. Implementare le seguenti funzioni:

- ▶ `struct entry *push(struct entry *s, int value);` aggiunge un nuovo valore value in cima allo stack.
- ▶ `struct entry *pop(struct entry *stack);` elimina l'elemento in cima allo stack corrente.
- ▶ `int top(struct entry *stack);` restituisce il valore in cima allo stack corrente.

## Esercizio 6 (/1)

Date le seguenti strutture:

```
1 struct node_t {  
2     int value;  
3     struct node_t *next;  
4 };  
5 struct list_t {  
6     struct node_t *head;  
7 };
```

Implementare le seguenti funzioni per la gestione di una lista non ordinata:

- ▶ `struct list_t *new_list()` che crea una nuova lista, inizialmente vuota;
- ▶ `void insert_on_head(struct list_t *list, int value)` che inserisce il numero `value` in testa alla lista;
- ▶ `void print(struct list_t *list)` che stampa, procedendo dalla testa verso la coda, il contenuto della lista;

## Esercizio 6 (/2)

- ▶ `void check_malloc(void *p)` che verifica la corretta allocazione della memoria e restituisce `exit(-1)` altrimenti;
- ▶ `void insert_on_tail(struct list_t *list, int value)` che inserisce il numero `value` in coda alla lista;
- ▶ `bool contains(struct list_t *list, int value)` che verifica se il numero `value` è presente nella lista;
- ▶ `void remove_from_list(struct list_t *list, int value)` elimina il nodo della lista, contenente il valore `value`; Nota: la lista può contenere valori duplicati, in questo caso uno solo dei nodi deve essere eliminato;
- ▶ `void empty_list(struct list_t *list)` svuota l'intera lista;
- ▶ `void free_list(struct list_t *list)` svuota l'intera lista e poi rimuove.

## Esercizio 7 (/1)

Date le seguenti strutture:

```
1 struct node_t {  
2     int value;  
3     struct node_t *next;  
4 };  
5 struct list_t {  
6     struct node_t *head;  
7 };
```

Implementare le seguenti funzioni per la gestione di una lista **ordinata**:

- ▶ `struct list_t *new_list()` che crea una nuova lista, inizialmente vuota;
- ▶ `void insert_into_list(struct list_t *list, int value)` che inserisce in modo “ordinato” il numero `value` all’interno della lista;



## Esercizio 7 (/2)

- ▶ `void print(struct list_t *list)` che stampa, procedendo dalla testa verso la coda, il contenuto della lista `list`;
- ▶ `void check_malloc(void *p)` che verifica la corretta allocazione della memoria e restituisce `exit(-1)` altrimenti;
- ▶ `bool contains(struct list_t *list, int value)` che verifica se il numero `value` è presente nella lista `list`;
- ▶ `void remove_from_list(struct list_t *list, int value)` elimina il nodo della lista `list`, contenente il valore `value`; Nota: la lista può contenere valori duplicati, in questo caso uno solo dei nodi deve essere eliminato;
- ▶ `void empty_list(struct list_t *list)` svuota l'intera lista `list`;
- ▶ `void free_list(struct list_t *list)` svuota l'intera lista `list` e poi rimuove `list`.